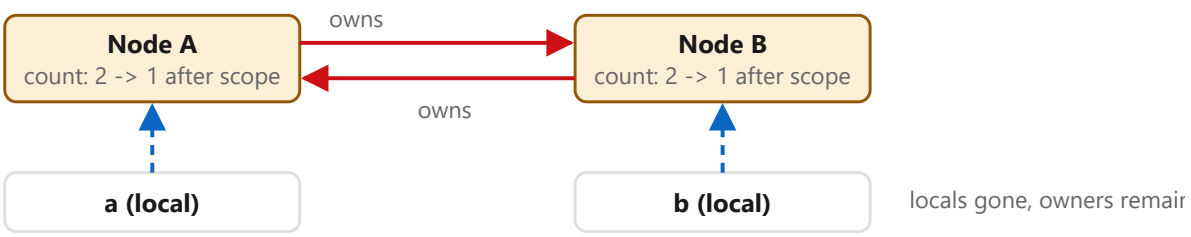
weak_ptr and Reference Cycles

Why `shared_ptr` cycles leak, how `weak_ptr` and `lock()` break them, and the three places a non-owning link belongs: parent links, caches and observers. Assumes shared_ptr.

The cycle problem

If `A` holds a `shared_ptr` to `B` and `B` holds one back to `A`, each keeps the other alive. When the local variables go away, both counts drop from 2 to 1, never to 0: **neither object is destroyed**. Compiled: both destructors never ran.



Each count stays at 1 after the variables die, so nothing deletes the pair.

```
struct Node { std::shared_ptr<Node> next; ~Node() { std::cout << "freed\n"; }
};
{
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();
    a->next = b;
    b->next = a;          // a cycle: prints nothing when the scope ends
}
```

LEAKS, NOT CRASHES A cycle never crashes. Memory just grows, and the destructors that would release files or sockets never run. Sanitizers with leak detection find it.

`weak_ptr` and `lock()`

A `weak_ptr` points at a shared object **without owning it**: it does not change the strong count, so it cannot keep the object alive. To use the object, ask for a temporary owner with `lock()`.



when the strong count reaches 0 the Widget is destroyed; the control block stays until the last weak_ptr goes.

weak_ptr observes; lock() turns it into an owner for as long as you need it.

```
std::shared_ptr<Widget> s = std::make_shared<Widget>();
// observe only: use_count() is still 1
std::weak_ptr<Widget> w = s;

// a shared_ptr, or null if the Widget is gone
if (auto sp = w.lock()) {
    // safe: sp keeps it alive inside this block
    sp->draw();
}
s.reset();                                  // the last owner leaves
// 1 (compiled): lock() would now return null
std::cout << w.expired();
```

CALL	MEANING
`w.lock()`	a `shared_ptr` that owns the object, or an empty one
`w.expired()`	true once every owner is gone (can change right after you ask)
`w.use_count()`	the number of owners
`w.reset()`	stop observing

ALWAYS LOCK, THEN TEST Never write `if (!w.expired()) w.lock()->use()`: the owner may vanish between the two calls. `lock()` once, test the result, use that.

Where weak_ptr belongs

Three places where a non-owning link is exactly right.

1. Parent links

A parent owns its children; a child only needs to find its parent. Make the back link weak and the tree frees itself.

```
struct Parent;
// back link: does not own
struct Child  { std::weak_ptr<Parent> parent; };
// forward link: owns
struct Parent { std::vector<std::shared_ptr<Child>> kids; };
// p.use_count() is 1 and the child is 2 (compiled): no cycle, both freed
```

2. Caches

A cache should hand out an object that is still in use, but not keep unused ones alive forever.

```
std::shared_ptr<Image> load(const std::string& key) {
    static std::map<std::string, std::weak_ptr<Image>> cache;
    // still in use: share it
    if (auto sp = cache[key].lock()) return sp;
    // otherwise load again
    auto sp = std::make_shared<Image>(key);
    cache[key] = sp;
    return sp;
}
```

3. Observers

A subject keeps weak pointers to its listeners, so a listener that goes away is simply skipped and never kept alive by the subject.

```
std::vector<std::weak_ptr<Listener>> subs;
for (auto& w : subs)
    // expired listeners are skipped
    if (auto sp = w.lock()) sp->notify();
// tidy up now and then
std::erase_if(subs, [](auto& w) { return w.expired(); });
```

USE	STRONG LINK	WEAK LINK
tree	parent to children	child to parent
cache	the users of an entry	the cache itself
observer	the listener's real owner	the subject's list

What a weak_ptr keeps alive

A `weak_ptr` keeps the **control block** alive, not the object. The counts have to outlive the object so that `expired()` can answer.

STATE	STRONG COUNT	WEAK COUNT	OBJECT	CONTROL BLOCK
all owners alive	1 or more	any	alive	alive
last owner gone, a weak_ptr remains	0	1 or more	destroyed	alive
last weak_ptr gone too	0	0	destroyed	freed

- With `make_shared` the object and the control block share one allocation, so the **memory of the object is not returned** until the last `weak_ptr` dies (the destructor still runs on time). For a huge object watched by a long-lived `weak_ptr`, `shared_ptr<T>(new T)` frees the object's memory earlier.
- `sizeof(weak_ptr)` is 16 bytes, like a `shared_ptr` (compiled).

- A weak pointer is for *observing* a shared object. For an object with a single owner use a raw pointer or reference to observe it, not a `weak_ptr`.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
weak_ptr and lock()	Effective Modern C++	Item 20, Use std::weak_ptr for shared_ptr-like pointers that can dangle	152 to 157